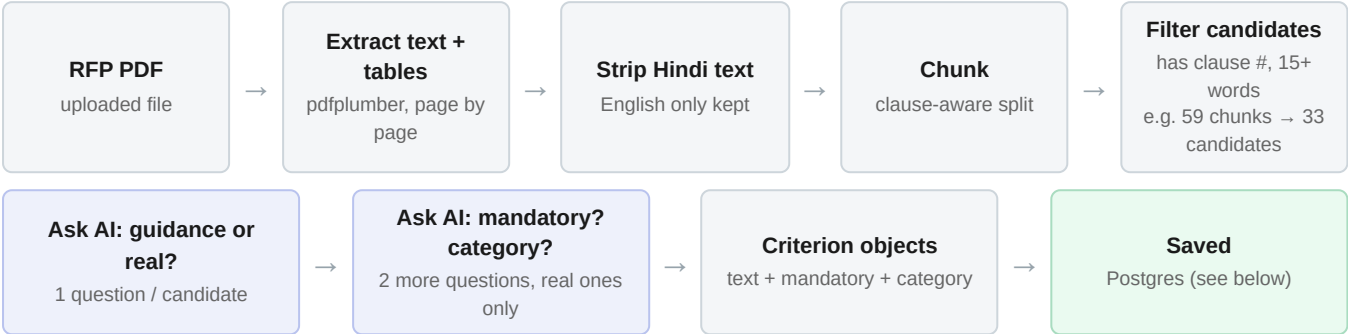


RFP Sentinel — RFP Criteria Extraction: How It Works & Today's Optimization

Covers the `extract_rfp_criteria()` step only — where an uploaded RFP PDF becomes a structured list of checkable requirements.

What actually happens, step by step



Separately, the same PDF's "Document required from seller" line is read directly (no AI call) into a `required_documents` list — this is a different question ("did the bidder submit the right document TYPES at all") from criteria checking ("does the content satisfy the requirement").

Where things actually live

What	Where it's stored
The uploaded RFP PDF itself	<code>data/rfps/{rfp_id}_{filename}</code> — plain file on disk
Extracted criteria (text, mandatory, category) and the 11 required-document names	Postgres — inside the LangGraph checkpoint for that RFP's evaluation run, keyed by <code>rfp_id</code> . Not in Qdrant at all.
Government norm documents (GTC, MSME policy, etc.) — the permanent rulebook	Qdrant , <code>norms</code> collection — chunked & embedded, shared across every RFP
A bidder's own submitted documents (technical + financial, sealed apart)	Qdrant , <code>bids</code> collection — chunked & embedded, tagged by <code>bid_id</code> + Packet I/II
What a bidder sees when they check "what documents do I need to submit"	Not a separate copy — <code>GET /bidder/rfps/{id}</code> reads the same <code>required_documents</code> list live out of Postgres, every time it's asked

Does old data get deleted when we re-run things? No, nowhere. Qdrant never hard-deletes: an old norm version gets marked superseded (kept, just excluded from active searches) rather than removed, and a bid only gets soft-deleted (`close_bid()` flags it `closed`) — an actual delete/purge step is intentionally not built yet. Postgres checkpoints for an RFP also just accumulate; nothing purges old ones automatically. Separately: today's repeated test runs (used to measure speed) called the extraction function directly and only printed results to the screen/log — they never went through the real save-to-Postgres path at all, so they didn't touch stored data either way.

Earlier approach vs. today's optimization

Piece	Before	After (today)
Mandatory / category / guidance-vs-real decision	Instant keyword guessing (regex), no AI involved	Asks the AI directly — slower, but more accurate and generalizes across RFP templates
Numeric threshold checks (e.g. "min. 10% discount")	A separate hand-written deterministic checker — built to work around the old small AI model getting these backwards	Removed — the new, larger remote AI model was confirmed to get these right on its own
How questions are sent to the AI	One at a time, strictly in sequence	Up to 5 at once (bounded — the server has a real concurrency limit, tested directly)

Network hiccups (server drops connection)	Whole run crashes immediately, all progress lost	Automatically retries a few times before giving up
Seeing what's happening / what went wrong	Nothing — only a final printout, or a raw crash trace	Every step logged live to logs/rfp_sentinel.log, with running "N of Total done" counters
Total time, this real 33-candidate RFP	~18 minutes	~5–8 minutes

Known open issue, not yet fixed: a short list of buyer-side "prohibited practices" disclaimers (page 12 of this RFP) is still sometimes wrongly accepted as real bidder criteria — the old regex approach caught this specific case correctly by remembering "once we see this block, everything after it is guidance too," a memory the new per-item AI check doesn't have yet. Tracked for a follow-up fix.

Project progress

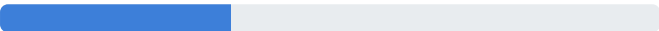
Buyer side (RFP → checks → publish) — ~95%



Bid evaluation side (ingest → check → score) — ~55%



Bidder-facing UI — backend only



Done:

- RFP upload, extraction, norm compliance check, human review + publish
- Bid ingestion with sealed technical/financial split
- Document-completeness & evidence-content checks
- Bidder login + read-only API (list RFPs, view checklist)

Not built yet:

- Deterministic scoring / ranking engine
- Bidder-facing screens, bid-upload UI, final review screen
- One connected pipeline (still separate scripts)

Overall v1 estimate: roughly 75–77% complete.